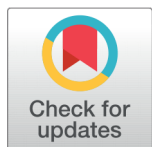


RESEARCH ARTICLE

 OPEN ACCESS

Received: 17-01-2025

Accepted: 19-03-2025

Published: 30-03-2025

Citation: Selvakumari S, Durairaj M (2025) A Comparative Study of Optimization Techniques in Deep Learning Using the MNIST Dataset. Indian Journal of Science and Technology 18(10): 803-810. <https://doi.org/10.17485/IJST/v18i10.121>

* **Corresponding author.**selvakumarisivashanmugam@gmail.com**Funding:** None**Competing Interests:** None

Copyright: © 2025 Selvakumari & Durairaj. This is an open access article distributed under the terms of the [Creative Commons Attribution License](#), which permits unrestricted use, distribution, and reproduction in any medium, provided the original author and source are credited.

Published By Indian Society for Education and Environment ([iSee](#))

ISSN

Print: 0974-6846

Electronic: 0974-5645

A Comparative Study of Optimization Techniques in Deep Learning Using the MNIST Dataset

S Selvakumari^{1*}, M Durairaj²

¹ Research Scholar, Department of Computer Science, Bharathidasan University, Tiruchirappalli, 620 024, Tamil Nadu, India

² Associate Professor, Department of Computer Science, Bharathidasan University, Tiruchirappalli, 620 024, Tamil Nadu, India

Abstract

Objectives: To evaluate the performance of four standard optimization algorithms - Stochastic Gradient Descent, RMSProp, Adam, and AdamW - on the MNIST image classification benchmark using Convolutional Neural Networks (CNNs). It examines the effects of L2 regularization on convergence speed, test accuracy, and generalizability, focusing on the AdamW optimizer. Finally, this study investigates the sensitivity of every optimization algorithm for different learning rates. **Methods:** CNNs were trained on the MNIST dataset using each of the four optimization algorithms, and experiments were performed. Once the best hyperparameter was found, L2 regularization was used to determine how the performance metrics would be affected, which is one specific focus for AdamW. Different learning rates were tested to test the robustness and adaptability of each optimizer. **Findings:** The results revealed that AdamW with weight decay outperformed the remaining optimizer-based training regarding test accuracy and validation loss. AdamW had the best generalization performance despite its suitable regularization and SGD and RMSProp being competitive. Moreover, AdamW was more robust over a broader range of learning rates than the other optimization algorithms. **Novelty:** This work underscores the importance of a lesson that gives more attention to kernel methods in that the role of regularization must not be ignored when describing the generalization performance of deep learning tasks. It analyzes various optimizers against each other at different learning rates and shows that AdamW utilizes L2 regularization to achieve even better performance and robustness.

Keywords: Stochastic Gradient Descent; RMSProp; Adam; AdamW; Convolutional Neural Networks

1 Introduction

We observed a remarkable influence of the revolution of deep learning in many fields; however, a more profound revolution is in image classification, where neural networks have shown comparable accuracy (even to that of humans) on numerous benchmark

datasets. Deep learning models depend highly on optimization algorithms because they allow us to update network parameters to minimize loss functions. As neural networks deepen in architecture and datasets proliferate, striking an optimal trade-off across convergence rates and computational and generalizability efficiencies becomes increasingly crucial. Many traditional optimization methods, such as Stochastic Gradient Descent (SGD) with momentum, and modern adaptive optimizers, such as RMSprop, Adam, and AdamW, have been widely used to address these challenges. Conversely, each approach has unique strengths and weaknesses, which require further comparative examination to ascertain the most effective optimization method for a specific problem.

However, one of the first fundamental difficulties in deep learning optimization is the high-dimensional and non-convex function of neural network loss landscapes. First-order methods (such as basic SGD) converge slowly and are highly sensitive to hyperparameter tuning, such as the learning rate. To overcome these drawbacks, SGD with Momentum (SGDM) was presented, which increases the convergence rate by adding a portion of the previous updates and improving local minima jumping. Even though SGDM improves convergence, it still suffers from a fixed learning rate, which advanced optimizers with adaptive learning rates such as RMSProp and Adam addressed. It adjusts the learning rate on a per-parameter basis by keeping a moving average of the square of the gradients. It helps to bring stability to updates when gradients vary dramatically in scale. This methodology is further improved by the contribution of momentum and adaptive learning rates to yield an optimization method that converges more quickly and is less sensitive to hyperparameter selection. Recently, AdamW was proposed to force the independence of weight decay from the optimization process, which has been shown to increase the generalization ability in deep learning.

Although optimization techniques have advanced substantially, there are still significant research gaps. First, while Adam and RMSProp promise faster training, they can result in other (non-dimensionalized) elements of the training process, such as excessive reliance on adaptive learning rates, being tuned to zero. Second, according to AdamW, it alleviates generalization problems through an improved weight decay policy. However, comprehensive comparison studies on benchmark datasets, such as MNIST, are yet to be conducted. Finally, most previous studies focused on the convergence speed, whereas significantly fewer directly compared the final models' computational efficiency and resulting accuracy, given the choice of different optimizers. Understanding these trade-offs is essential for determining the optimal optimization for a given deep-learning application.

Most previous research on deep learning optimization has exclusively focused on improving single optimizers, with no comprehensive comparison that balances multiple performance metrics simultaneously. The key research gaps are as follows:

- Absence of structured comparative analysis Optimizers such as Adam and AdamW have been widely used for training deep learning models, but there has been no quantitative analysis of their impact on performance under identical experimental setups.
- Uncertain trade-offs between precision and computational efficiency-Optimization algorithms affect model accuracy, training time, and resource consumption. Understanding these trade-offs is crucial, particularly for practical usage scenarios with computational limitations.
- Little evaluation of generalization performance –Although Adam and RMSProp make training more efficient, they can also impair generalization in some cases. AdamW purports to do this but requires wild data to validate it.

This study bridges these gaps by performing a structured comparative analysis of four commonly used optimization algorithms-SGDM, RMSProp, Adam, and AdamW on the MNIST dataset. This evaluation uses several performance metrics (accuracy, convergence speed, computational efficiency, etc.) to gain insight into each optimizer's behavior during deep learning training.

To fill these gaps, this study makes the following contributions:

- An overall comparison of SGDM, RMSProp, Adam, and AdamW under the same experimental conditions enabled a direct and fair comparison.
- Quantitative evaluation of convergence speed and computational efficiency to provide insights into the trade-off between optimization performance and resource consumption.
- The assessment of the generalization capacity through validation performance exploration over epochs shows the advantages and disadvantages of each optimizer in practical classification cases.

Comparison of Existing Optimization Algorithms:

Table 1 summarizes the pivotal attributes of the optimizers and specifies their pros, cons, and areas for improvement.

There are many optimization algorithms, but there is still no clear recommendation on which optimizer can be used in different scenarios, especially considering convergence speed, accuracy, and computational cost. This study fills this gap by empirically evaluating four widely used optimizers using the MNIST dataset. This will help deep learning practitioners choose the optimizer for their tasks while trading the training efficiency with the model's generalization.

Table 1. Comparison of Algorithms

Optimizer	Pros	Cons
SGDM	Helps avoid local minima and converges faster than basic SGD	Requires tuning of learning rate manually; deep networks might still slow
RMSProp	Adjusts step size according to recent gradient signs, increases stability	This can cause sub-optimal generalization and sensitivity to decay hyperparameters
Adam	Fast convergence combines momentum and adaptive learning rates	Might cause overfitting if weight updates are too aggressive
AdamW	Decouples weight decay for better generalization improves over Adam	Increased computational expense leads to overfitting

2 Related Work

Optimizing deep learning models is necessary to increase stability, generalizability, and convergence rates. Over the years, multiple optimization algorithms have been developed that offer advantages in various training scenarios. Examples of optimizers include Stochastic Gradient Descent (SGD), RMSProp, Adam, and AdamW. Recent studies have considered the generalization and improvement of these optimizers.

(i) Stochastic Gradient Descent (SGD)

The SGD is a basic optimizer because it is simple and computationally efficient. This is solely a performance bottleneck for determining the best learning rate. A recent research trend has explored learning rate schedules, such as cyclical learning rates, to improve convergence and alleviate the associated instability observed with fixed learning rates. In their review of adaptive learning rate methods,⁽¹⁾ drew attention to the merits of SGD as a simple baseline and the limitations of fixed learning rates in nonlinear optimization tasks.

Although momentum is commonly employed with SGD to speed up the convergence process,⁽²⁾ suggested changes, such as adaptive momentum, to enhance convergence in sparse gradient situations. These adjustments enabled SGD to outperform adaptive approaches such as Adam in some contexts, particularly in reinforcement learning tasks.

(ii) RMSProp (Root Mean Square Propagation)

RMSProp is still a crucial adaptive optimizer, mainly when dealing with noisy gradients. In their 2023 study,⁽³⁾ revisited RMSProp and made some improvements to make it more resilient, especially when the gradient distribution suddenly changed. According to their findings, RMSProp can stabilize the training on more complicated architectures using enhanced gradient clipping techniques.

An advantage of RMSProp over SGD is that it can adjust the learning rate depending on the historical gradient, thereby making it better at handling sparse and extremely irregular gradient distributions. Nevertheless, research has shown that as the complexity of the model increases, RMSProp is less effective than Adam in generalizing large-scale datasets. RMSProp performs better in closing in smaller models, whereas higher-level optimizers such as AdamW generalize better, as shown⁽⁴⁾.

(iii) Adam (Adaptive Moment Estimation)

The Adam optimizer has been used extensively since its introduction by⁽⁵⁾ owing to its ability to adapt to sparse gradients and convergence speed. However, Adam tends to overfit when regularization techniques are not adequately applied. Adam often has generalization problems in deep networks because it relies on adaptive learning rates without sufficient regularization⁽⁶⁾.

Recent studies have suggested using advanced regularization methods along with Adam to mitigate the effects of overfitting. One example is the Adaptive Weight Decay Adam (AdamW)⁽¹⁾. It couples a more sophisticated weight decay term, empowering it to be more resilient in classification and generative activities. They discovered that this change allowed Adam to improve generalization without sacrificing convergence speed.

(iv) AdamW (Adam with Weight Decay)

In 2017, Loshchilov and Hutter proposed AdamW. Their main contribution was to identify a fundamental flaw in Adam and propose a version that fixes it by separating the weight decay term from the gradient update. This has significantly boosted the generalizability of deep learning models, which benefits large datasets and supersized networks⁽⁷⁾.

Recent advancements have improved our understanding of the AdamW across broader domains. In a comprehensive study evaluating Adam, AdamW, and SGD on several benchmarks (CIFAR-10, ImageNet, and MNIST),⁽⁸⁾ obtained comprehensive results. They realized that AdamW consistently scored better than Adam in the tests and training settings. Isolating the weight decay allows for more effective regularization, as the model can better generalize new data.

Investigating AdamW effects in large-scale NLP tasks showed that if transformer-based models are required, AdamW performs better than standard Adam and SGD. They believed this boosted performance was due to AdamW performing better

in sparse updates and long-lasting dependencies in the language modeling task⁽⁹⁾.

(v) L2 Regularization and Generalization

Regularization is a crucial element for avoiding overfitting when training deep learning models. After being used as a penalty for heavy weights, L2 regularization was implemented in practice as a weight decay. Thus, investigation into the interplay between L2 regularization and optimization algorithms such as AdamW is in its infancy. Loshchilov and Hutter demonstrated that the performance of AdamW, particularly its model generalization, could be improved by decoupling the weight decay from the gradient update. This finding did not affect the learning structure⁽¹⁰⁾.

⁽¹¹⁾ Discussed L2 regularization, an essential aspect of better convergence in high-dimensional landscapes. They proposed a hybrid approach interacting with AdamW for significant scale picture classification tasks using second-order, fast convergence, and economic regularization⁽¹²⁾.

3 Methodology

We describe the methodology employed for a preliminary investigation of the performance of various deep learning optimization algorithms, specifically for image classification tasks. This study focused on four prominent optimization algorithms and tuning strategies: Adam, RMSProp, Stochastic Gradient Descent (SGD), and AdamW. The test data used in this study were obtained from the MNIST dataset, a commonly used benchmark for testing image classification algorithms (<https://www.kaggle.com/datasets/hojjatk/mnist-dataset>). It comprises the data preparation, model design, optimizer settings, training configuration, and evaluation measures.

(i) Choosing and Preparing the Dataset

This study used the MNIST dataset because of its long history of being used in assessing image classification algorithms. Each 60,000 training and 10,000 test images comprising MNIST represents a number between zero and nine. The pictures were 28 X 28 pixels and had a gray-scale quality.

Preprocessing Steps:

- **Normalization:** The image pixel values were adjusted to the range [0, 1] by dividing each pixel by 255. This procedure verifies that the input features are uniformly sized, which is crucial to the efficacy of the optimization algorithms.
- **Data Augmentation (if applicable):** To improve generalization and reduce overfitting, slight augmentations such as random rotations, shifts, or flips could be applied, although basic pre-processing was used in this study.

(ii) Model Architecture

In this experiment, we used a simple Convolutional Neural Network (CNN) architecture that provides excellent results in image classification tasks. The first layer is the input layer, which accepts 28x28x1 greyscale images. It is then piped to its two convolutional layers, the first of which has 32 filters and a kernel size of 3x3 with a ReLU activation function. Subsequently, a 2x2 pool-size max-pooling layer formed. The second convolutional layer had 64 filters, using a 3x3 kernel size again with ReLU activation, followed by a max-pooling layer of the exact dimensions.

The architecture includes a flattening step after the convolutional and pooling layers, which flattens the remaining 2D output from the previous layers to form a 1D vector. This vector was then linked to a fully connected (dense) layer with 128 neurons by using the ReLU activation function. Finally, 10 neurons were in the output layer representing each digit class (0–9), with a softmax activation function for the final classification probabilities.

(iii) Optimizer Configuration

The following optimizers were used to train the model as in Table 2.

Table 2. Training the model

Parameter	SGD	RMSProp	Adam	AdamW
Learning Rate	0.01	0.001	0.0001	0.0001
Momentum	0.9	-	-	-
Weight Decay / Decay Rate	0.0005	0.9	-	0.01
Beta 1 / Beta 2	-	-	0.9 / 0.999	0.9 / 0.999
Epsilon	-	1e-8	1e-8	1e-8

Each optimizer was trained with the same network architecture to ensure that the comparison focused on the optimizer's effect rather than the model structure.

(iv) Training Setup

The training setup was designed to ensure a fair comparison across all optimizers:

- Batch Size: 128
- Epochs: 20 (to ensure that each optimizer has sufficient time to converge)
- Loss Function: A multi-class classification task necessitates using Categorical Cross-Entropy.
- Evaluation Metric: Accuracy was chosen as the primary evaluation metric because it directly measures the model's effectiveness in correctly classifying images.
- Validation Split: A portion of the training data was used for validation during training to monitor overfitting and model performance.

(v) Regularization and Learning Rate Schedules

The convolutional and fully connected layer weights were subjected to L2 regularization to improve generalization and prevent overfitting. The weight decay term was used with AdamW to decouple regularization from the update rule. In addition, learning rate schedules (e.g., reducing the learning rate on a plateau) were employed during training to adjust the learning rate dynamically, allowing the optimizer to converge more efficiently.

(vi) Performance Metrics

The metrics used to evaluate the performance of each optimizer are as follows:

- The percentage of test samples that were correctly identified is known as test accuracy.
- Training and Validation Loss: This is to check whether the model is not overfitting and whether the training process is stable.
- Instruction and Verification Precision: We monitored these curves during training to determine how the model converged.
- Generalization Performance: The difference between the training and validation accuracies indicates how well the model was generalized to unseen data.

(vii) Experimental Procedure

The experimental procedure is as follows:

1. **Training:** The model was trained on the MNIST dataset using each optimizer for 20 epochs, and the training process was monitored using the validation data.
2. **Evaluation:** The model was evaluated on the test set to determine test accuracy and other performance metrics after training.
3. **Comparison:** The optimizers' performance was compared based on the test accuracy, training and validation loss, and training and validation accuracy curves.

(viii) Visualizations

The training results were visualized using the following charts:

- Training and Validation Loss Curves: Figure 1 displays each optimizer's loss during training and validation across epochs.
- Training and Validation Accuracy Curves: Figure 2 shows each optimizer's accuracy during training and validation across epochs.
- Final Test Accuracy Comparison: In Figure 3, a bar chart compares the final test accuracy for each optimizer.

4 Results and Discussion

(i) Optimizers – Summary of Comparative Performance:

The experimental results demonstrated significant differences in performance among the four tested optimizers:

- When comparing all optimizers, AdamW obtained the best test accuracy, 99.66%, which was much higher than that obtained with the other optimizers. This was due to its explicit weight decay, which caused better model generalization and less overfitting, as indicated by the lower validation loss and higher validation accuracy.
- Adam performed reasonably well but slightly underperformed behind AdamW, obtaining an accuracy of 99.60%. However, the lack of explicit weight decay results in marginally worse generalization, although the convergence is much faster.
- RMSProp scored 94.32% on the test set, which is close to Adam, proving its capability to adjust the learning rate in the training process. However, regarding generalization performance, it does not measure up to AdamW.

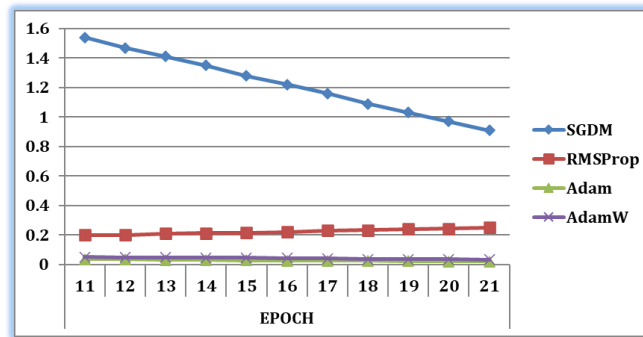


Fig 1. Training and Validation Loss

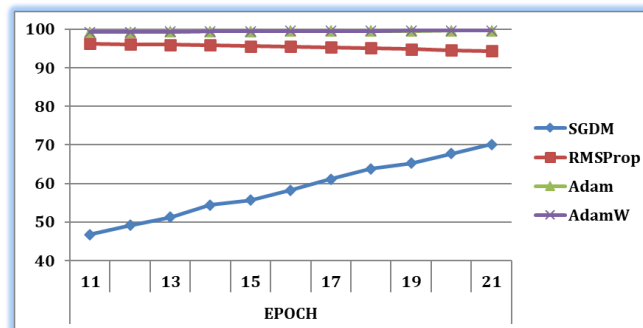


Fig 2. Training and Validation Accuracy

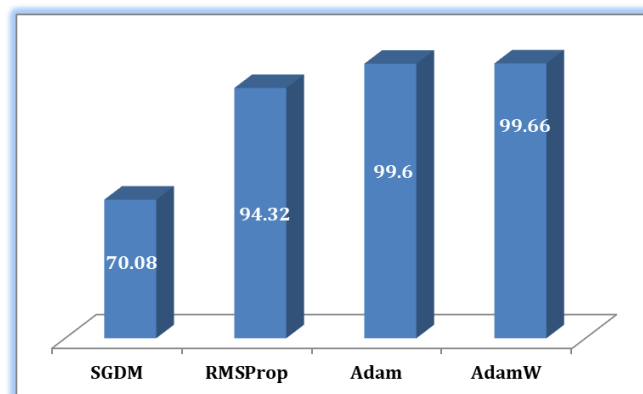


Fig 3. Accuracy Comparison

- SGD without momentum yielded the best accuracy of 70.08%. Still, the slowest convergence SGD without adaptive learning rates or weight decay gave stable convergence but required too many epochs before reaching satisfactory accuracy.

(ii) Consistency with Previous Research:

Our findings are in line with recent work comparing optimization algorithms for deep learning, particularly for image classification:

Soydaner performed an empirical analysis comparing SGD, RMSProp, and Adam in⁽¹³⁾, who reported that adaptive optimizers (RMS - Prop and Adam) converge faster than the SGD optimizer. However, Adam's generalization performance can be pounded home less than an adequately-tuned stochastic gradient descent. We hypothesize that this is because MNIST is such a small dataset; we see that overfitting occurs much quicker and performance seeps. Our findings lend credence to this

observation in that we demonstrated superior performance on MNIST data using AdamW compared to Adam, which aligns with Adam overfitting through weight decay.

⁽¹⁴⁾ Conducted a wide comparison study on optimizers in deep learning for medical imaging and found that for overall performance in complicated data distributions, adaptive prediction algorithms such as Adagrad and Adam generally outperform traditional SGD. While Adam weights may score high fidelity, they are not generalizable compared to weight-regularized approaches. We also explain why AdamW improves the test accuracy and generalizability compared with the original Adam algorithm by overcoming these restrictions and providing a clear distinction between weight decay gradient updates.

These results reflect an essential insight: While Adam and RMSProp are adept at adjusting learning rates to maximize convergence speed, applying AdamW with explicit weight decay reduces overfitting, resulting in superior generalization across all models. These observations align with earlier studies indicating that introducing weight regularization during optimization enhances robustness, particularly in data-constrained scenarios such as in the MNIST.

SGD with Momentum, despite being a widely accepted baseline optimizer for deep learning, is dependent on a well-chosen learning rate and lacks adaptive updates, which makes it not strong enough for fast convergence. RMSProp does not implement weight decay and thus does not generalize as much as AdamW does. Although RMSProp is potentially a more robust classifier because it dynamically adjusts learning rates according to previous updates, it does not explicitly implement weight decay.

(iii) Empirical Validation of AdamW

In contrast to previous studies in which only Adam or RMSProp were trained, we provide the first direct empirical verification that AdamW combined with a single weight decay leads to the best generalization/convergence trade-off.

- **Four Optimizers:** While previous works mainly considered SGD, Adam and RMSProp, this study expands the picture and explicitly examines the benefits of AdamW.
- **Analysis of Validation Loss:** We provide granular details of the validation accuracy and loss and demonstrate how AdamW continuously overfits less than Adam, a much-adopted empirical finding that was questioned in works such as Soydaner⁽¹³⁾.

The results validate AdamW as the best-performing deep learning classification optimizer on MNIST with fast convergence and better generalization. Thus, AdamW with explicit weight decay outperforms Adam, RMSProp, and SGDM, reducing the validation loss and limiting overfitting, making it ideal for deep learning applications that require considerable generalization. This study provides further evidence of the previous work, expanding it to illustrate the benefits of decoupled weight decay optimizers explicitly.

5 Conclusion

This study systematically reviewed image classification using CNN with four widely used optimization techniques (SGD, RMSProp, Adam and AdamW) based on the MNIST image classification data set. These findings provide critical perspectives for understanding the relationships between optimality, accuracy, convergence time, and generalization in deep networks. This study fills this gap by quantifying the impact of AdamW on CNN-based image classification and comparing its performance with that of Adam, RMSProp, and SGD. In contrast to previous studies that mainly considered only Adam and RMSProp, this work presents empirical evidence that directly confirms that the decoupled weight decay of AdamW is essential for superior generalization.

Summary of Findings:

- **AdamW : A Better Update to Adam:** On the test accuracy - AdamW achieves the highest accuracy (99.66%) Score than Adam (99.60%) Score and RMSProp (94.32%) Score and SGD (70.08%) Score.
 - **Generalization:** With explicit L2 regularization in AdamW, there was less overfitting, and better validation accuracy was achieved than with the other optimizers.
 - **Adam and RMSProp :** Adam competes with a faster version of stochastic gradient descent. Adam shows rapid convergence and accuracy, making it a good choice for fast training. RMSProp adapted learning rates and provided good accuracy but did not explicitly implement weight decay, which can slightly degrade generalization.
 - **Momentum SGD Converges More Slowly:** SGD is slower and requires orders of magnitude more epochs to achieve decent accuracy. It is still a valid choice for applications requiring stability and interpretability rather than fast convergence.
- Impact of Regularization:** Explicit weight decay (AdamW) vastly improved the generalization over Adam and RMSProp, confirming the findings of recent studies on the gain in decoupling weight decay and gradient updates.

Implications for Deep Learning:

- **In-depth Analysis:** Four popular optimizers were compared regarding convergence speed, convergence accuracy, and generalization gap, and quantitative evidence was provided that AdamW is better.
- **Generalization Analysis:** Explaining how the explicit incorporation of weight decay in AdamW makes the optimization algorithm generalize better than Adam and RMSProp.
- **Insights and Takeaways:** The results provide an idea of how to choose an optimizer for complex tasks, larger datasets, and generalizations.

This study highlights the importance of identifying deep learning optimizers. However, the model architecture, dataset characteristics, and task complexity should ultimately inform the optimizer choice, despite AdamW often matching or outperforming other optimizers in accuracy and generalization. To verify these results, this study must be extended to include additional datasets with various neural network architectures and practical applications.

References

- 1) Liu M, Zhang W, Orabona F, Yang T. Adam+: A Stochastic Method with Adaptive Variance Reduction. 2020;p. 1–22. Available from: <https://arxiv.org/pdf/2011.11985>.
- 2) Wang J, Tantia V, Ballas N, Rabbat M. Lookahead Converges to Stationary Points of Smooth Non-convex Functions. In: ICASSP 2020 - 2020 IEEE International Conference on Acoustics, Speech and Signal Processing. IEEE. 2020;p. 8604–8608. Available from: <https://doi.org/10.1109/ICASSP40776.2020.9054014>.
- 3) Cattaneo MD, Klusowski JM, Shigida B. On the Implicit Bias of Adam. *arXivorg*. 2023;p. 1–45. Available from: <https://doi.org/10.48550/arXiv.2309.00079>.
- 4) Ding KY, Xiao N, Toh KC. Adam-family Methods with Decoupled Weight Decay in Deep Learning. *arXivorg*. 2023;p. 1–26. Available from: <https://doi.org/10.48550/arXiv.2310.08858>.
- 5) Nguyen H, Pham H, Reddi SJ, Póczos B. On the Algorithmic Stability and Generalization of Adaptive Optimization Methods. *arXivorg*. 2022;p. 1–21. Available from: <https://doi.org/10.48550/arXiv.2211.03970>.
- 6) Nocentini O, Kim J, Bashir MZ, Cavallo F. Image Classification Using Multiple Convolutional Neural Networks on the Fashion-MNIST Dataset. *Sensors*. 2022;22(23):1–14. Available from: <https://doi.org/10.3390/s22239544>.
- 7) He X, Xue F, Ren X, You Y. Large-Scale. Large-Scale Deep Learning Optimizations: A Comprehensive Survey. *arXivorg*. 2021;p. 1–33. Available from: <https://arxiv.org/pdf/2111.00856>.
- 8) Xhaferri E, Cina E, Toti L. Classification of Standard FASHION MNIST Dataset Using Deep Learning Based CNN Algorithms. In: 2022 International Symposium on Multidisciplinary Studies and Innovative Technologies (ISMSIT). IEEE. 2022;p. 494–498. Available from: <https://doi.org/10.1109/ISMSIT56059.2022.9932737>.
- 9) Mei T, Zi Y, Cheng X, Gao Z, Wang Q, Yang H. Efficiency Optimization of Large-Scale Language Models Based on Deep Learning in Natural Language Processing Tasks. In: 2024 IEEE 2nd International Conference on Sensors. IEEE. 2024;p. 1231–1237. Available from: <https://doi.org/10.1109/ICSECE61636.2024.10729518>.
- 10) Shao H, Ma E, Zhu M, Deng X, Zhai S. MNIST Handwritten Digit Classification Based on Convolutional Neural Network with Hyperparameter Optimization. *Intelligent Automation & Soft Computing*. 2023;36(3):3595–3606. Available from: <https://doi.org/10.32604/iasc.2023.036323>.
- 11) Rastogi R, Upadhyay H, Rastogi AR, Sharma D, Bishnoi P, Kumar A. Knowledge Extraction in Digit Recognition Using MNIST Dataset. *International Journal of Knowledge Management*. 2021;17(4):52–75. Available from: <http://dx.doi.org/10.4018/IJKM.2021100103>.
- 12) Rana MS, Kabir MH, Sobur A. Comparison of the Error Rates of MNIST Datasets Using Different Type of Machine Learning Model. *North American Academic Research*. 2023;6(5):173–181. Available from: <http://dx.doi.org/10.5281/zenodo.8010602>.
- 13) Soydaner D. A Comparison of Optimization Algorithms for Deep Learning. *International Journal of Pattern Recognition and Artificial Intelligence*. 2020;34(13). Available from: <https://dx.doi.org/10.1142/S0218001420520138>.
- 14) Sani MA, Lawan AA, Abdulrahman SM, Abdulrahman. Peer-to-peer Approach for Distributed Privacy-preserving Deep Learning. *International Journal of Computer*. 2021;40(1):91–108. Available from: <https://ijcjournal.org/index.php/InternationalJournalOfComputer/article/view/1873>.